

Connector implementation plan for HubSpot, Stripe, Calendly, and Apollo

Below is a practical, prioritized implementation plan for building robust native connectors to HubSpot, Stripe, Calendly, and Apollo. Each connector section covers purpose, auth and security, data model and mapping, events and webhooks, integration flows, error handling and rate limits, testing and QA, privacy and compliance, monitoring, and rough effort estimate. At the end you'll find a recommended rollout sequence, owners, and a 12-week integration sprint map.

Common integration principles

- Use official APIs and SDKs for reliability and compliance.
- OAuth 2.0 for user-level connectors where supported; API keys for server-to-server where appropriate.
- Webhooks first for real-time events; polling only as fallback.
- Idempotency for all write operations.
- Per-tenant quotas and backoff to protect platform and third-party limits.
- Feature flags to gate rollout and enable quick rollback.
- Audit logs and trace IDs for every sync and webhook event.
- Secure secrets in vaults and rotate keys regularly.
- Privacy controls surfaced to customers for data retention and deletion.

HubSpot Connector

Purpose

Sync inbound intelligence and actions into marketing and sales workflows: create contacts, log mentions as engagements, push alerts as tasks, and pull CRM data for enrichment and segmentation.

Auth and security

- OAuth 2.0 with HubSpot scopes for contacts, engagements, CRM objects, and webhooks.
- Store refresh tokens securely and implement token refresh flow.
- Support per-workspace installation and multi-account linking.

Data model and mapping

- Datiq Mention → HubSpot Engagement: map mention text to engagement body; include source, sentiment, URL, and tags.
- Author → Contact: map handle, name, email (if available), social handles, reach metric to

HubSpot contact fields.

- Entities/Topics → Contact or Company properties: custom properties for brand signals and last mention timestamp.
- Alerts → Tasks: create tasks assigned to owner with link to mention.

Events and webhooks

- Inbound: HubSpot webhooks for contact updates and form submissions if Datiq needs to react.
- Outbound: Datiq sends webhooks to HubSpot via Engagements API and Contacts API. Use idempotency keys.

Integration flows

1. Install: OAuth flow; request required scopes.
2. Mapping UI: allow users to map Datic fields to HubSpot properties and choose default owner.
3. Sync rules: dedupe by email or social handle; configurable frequency.
4. Alert actions: create task or ticket when rule triggers.
5. Backfill: optional historical push for selected mentions.

Error handling and rate limits

- Respect HubSpot rate limits; implement exponential backoff and queueing.
- Surface failed syncs in UI with retry button and error reason.
- Log webhook signature verification failures.

Testing and QA

- Unit tests for mapping logic.
- Integration tests using HubSpot developer accounts.
- End-to-end tests for OAuth, webhook verification, and task creation.

Privacy and compliance

- Allow customers to opt out of pushing PII.
- Provide deletion endpoint to remove synced contacts on request.

Monitoring

- Metrics: webhook delivery success, sync latency, failed sync rate, API quota usage.

Rough effort estimate

- Medium: 3–5 developer sprints (4–8 weeks) including UI mapping and backfill.

Stripe Connector

Purpose

Enable tip jar payments, payouts, transaction reconciliation, and revenue reporting. Provide secure payment flows and webhook-driven transaction lifecycle handling.

Auth and security

- Stripe Connect for platform payouts to creators. Use OAuth for connected accounts or direct API keys for platform-managed flows.
- PCI compliance via Stripe Checkout or Stripe Elements; never store card data.
- Use webhooks with signature verification.

Data model and mapping

- Payment → Transaction record: map Stripe charge, payment intent, fees, net payout, currency, and metadata (datiq_page_id, user_id).
- Payout → Payout record: map payout id, status, arrival date.

- Refund → Refund record: map refund id, amount, reason.

Events and webhooks

- Subscribe to `payment_intent.succeeded`, `charge.refunded`, `payout.paid`, `payment_intent.payment_failed`.
- Webhook handler must verify signature and update transaction state atomically.

Integration flows

1. Checkout: embed Stripe Checkout or Elements on Datic pages for tip jar.
2. Metadata: attach Datic identifiers to payment metadata for reconciliation.
3. Webhook processing: update transaction status, notify user, and trigger payout workflows.
4. Payouts: for Connect accounts, manage onboarding and KYC flows via Stripe Connect.

Error handling and rate limits

- Handle payment failures gracefully with retry and user messaging.
- Implement idempotency keys for payment creation.
- Monitor webhook retries and dead-letter queue for manual review.

Testing and QA

- Use Stripe test keys and simulate webhooks.
- Test SCA flows for EU customers.
- Validate reconciliation reports.

Privacy and compliance

- KYC and tax reporting considerations for payouts.
- Provide receipts and transaction export for creators.

Monitoring

- Metrics: payment success rate, failed payments, payout latency, dispute rate.

Rough effort estimate

- Medium: 3–6 developer sprints (4–10 weeks) depending on Connect and KYC complexity.

Calendly Connector

Purpose

Embed booking flows on Datic pages and capture scheduled events into Datic workflows and CRM connectors.

Auth and security

- OAuth 2.0 with Calendly scopes for scheduling and webhooks.
- Store access tokens securely and refresh as needed.

Data model and mapping

- Event → Booking record: map event type, start/end, invitee name, email, timezone, and custom questions.
- Availability → UI: fetch event types and availability for embedding.

Events and webhooks

- Subscribe to Calendly webhooks for `invitee.created` and `invitee.canceled`.
- Webhook payload includes invitee details and event URI.

Integration flows

1. Install: OAuth flow to connect Calendly account.
2. Embed: provide embeddable widget or redirect link on DatIQ page.
3. Capture: on `invitee.created`, create a DatIQ booking record and optionally push to HubSpot or send notification.
4. Sync: allow mapping of Calendly event fields to CRM properties.

Error handling and rate limits

- Respect Calendly rate limits; implement retry and idempotency for webhook processing.
- Validate webhook signatures.

Testing and QA

- Use Calendly sandbox or test accounts to simulate bookings and cancellations.
- Test timezone handling and DST edge cases.

Privacy and compliance

- Ensure invitee PII is stored with consent and retention controls.

Monitoring

- Metrics: bookings per page, webhook delivery success, embed load times.

Rough effort estimate

- Low to medium: 2–4 developer sprints (3–6 weeks).

Apollo Connector

Purpose

Enrich leads and enable outbound sales workflows by pushing DatIQ-captured leads and signals into Apollo for prospecting and sequences.

Auth and security

- API key or OAuth depending on Apollo offering. Use per-tenant credentials stored securely.
- Respect Apollo API usage policies.

Data model and mapping

- Datiq lead → Apollo prospect: map name, company, title, email, social handles, enrichment data, and source URL.
- Signals → Apollo tags: tag prospects with topics, sentiment, and last mention.

Events and webhooks

- Outbound: Datiq calls Apollo APIs to create or update prospects and add to lists.
- Inbound: if Apollo supports webhooks for sequence events, subscribe to update prospect status.

Integration flows

1. Connect: user provides API key or OAuth.
2. Mapping UI: map Datiq fields to Apollo prospect fields and choose lists/sequences.
3. Lead push: on mention with lead intent, create prospect and optionally start sequence.
4. Sync status: poll or receive callbacks for sequence progress and update Datiq lead status.

Error handling and rate limits

- Implement batching for bulk prospect creation.
- Handle duplicate detection using email or domain heuristics.
- Backoff on rate limit responses.

Testing and QA

- Use Apollo sandbox or test account to validate prospect creation and sequence enrollment.
- Test dedupe logic and mapping edge cases.

Privacy and compliance

- Ensure consent for adding contacts to outbound sequences; provide unsubscribe and data deletion flows.

Monitoring

- Metrics: prospects created, sequence starts, bounce rates, API error rates.

Rough effort estimate

- Medium: 3–5 developer sprints (4–8 weeks).

Prioritized rollout and sequencing

Priority order

1. Stripe — enables monetization and immediate revenue; high business impact.
2. Calendly — increases utility for SMBs and creators; relatively low effort.
3. HubSpot — high enterprise value; more complex mapping and UI.
4. Apollo — strategic for sales teams; medium complexity.

Rationale

- Payments unlock revenue and creator adoption quickly.
- Calendly is low friction and increases conversion on Daticade pages.
- HubSpot integration drives enterprise adoption and retention.
- Apollo supports sales workflows and upsell opportunities.

12-week integration sprint map with owners and milestones

Week 0 Planning

- Finalize API keys, legal review, security checklist, and design mapping UI. Owners: PM, Tech Lead, Security.

Weeks 1–3 Stripe

- Implement Checkout/Elements, metadata mapping, webhook handler, test payments, Connect onboarding. Owners: Backend, Frontend, DevOps. Deliverable: Tip jar live in sandbox.

Weeks 4–5 Calendly

- OAuth, embed widget, webhook handling for invitee.created, booking UI. Owners: Backend, Frontend. Deliverable: Booking embed and booking record sync.

Weeks 6–9 HubSpot

- OAuth, mapping UI, engagement creation, contact dedupe, webhook verification, backfill tool. Owners: Backend, Frontend, Data Eng. Deliverable: CRM sync and task creation.

Weeks 10–12 Apollo

- API key flow, mapping UI, prospect creation, sequence enrollment, batching and dedupe. Owners: Backend, Frontend. Deliverable: Lead push and sequence trigger.

Parallel tasks across sprints

- QA, security review, monitoring dashboards, documentation, and marketing collateral.

Operational and support considerations

- Support playbooks for connector failures and common errors.
- Admin UI for viewing connector status, recent webhook deliveries, and manual retries.
- Billing and limits: surface usage and warn customers before hitting quotas.
- SLA and enterprise options: offer higher quotas and dedicated support for Enterprise tier.

Quick wins and low effort features to add early

- Connector status dashboard showing last successful sync and error counts.
- One-click test webhook for each connector to validate setup.
- Prebuilt mapping templates for HubSpot and Apollo to reduce setup friction.
- Retry queue UI for failed webhook events.

Final recommendations

1. Start with Stripe and Calendly to unlock monetization and immediate user value.
2. Ship robust webhook handling and idempotency before enabling write operations to third parties.
3. Expose mapping UI and audit logs so non-technical users can control how data flows.
4. Instrument monitoring and alerts for connector health and quota usage.
5. Bundle connectors into Growth and Pro tiers to drive upgrades.

If you want, I will now produce detailed API contract examples and sample webhook payload handlers for each connector, plus a Jira-ready task breakdown for the Stripe and HubSpot workstreams. Which deliverable should I prepare next?